CS-515 Advanced Computer Science Practicum

Data Structures and Algorithms

T26049 — Yashank

Algorithms Performed

Normal Matrix Multiplication

Core Implementation: The standard $O(N^3)$ row-by-column iterative multiplication approach used as the foundational baseline for performance testing.

WHAT WAS ADDED

Algorithm-level multiprocessing. Introduced concurrent execution blocks (via `concurrent.futures`) to chunk the matrix rows and distribute the workload across multiple distinct CPU cores simultaneously for matrices larger than 128 × 128.

WHAT WAS REMOVED

Removed the strictly sequential, single-core processing loop for large matrices. By chunking the workload, we bypassed the single-thread bottlenecks and Python's Global Interpreter Lock (GIL) to force maximum hardware utilization.

Normal Stressan Matrix Algorithm

Core Implementation: The pure, textbook divide-and-conquer approach that computes 7 intermediate matrices to achieve $O(N^{2.81})$ time complexity, strictly recursing all the way down to a 1×1 base case.

WHAT WAS ADDED

Unconditional power-of-2 zero padding. Because pure Strassen requires perfect halving at every recursive step, wrapped the algorithm in logic that forces any arbitrarily sized matrix (e.g., 500 × 500) up to the nearest perfect power of 2 (512 × 512) before execution, stripping the zeroes off at the end.

WHAT WAS REMOVED

Removed manual dimension constraints, meaning the algorithm no longer crashes on non-perfect square inputs. However, explicitly did not remove the deep recursion, deliberately retaining the algorithm's massive addition-operation bloat to benchmark its theoretical limits against hardware reality.

Hybrid Matrix Multiplication

Core Implementation: A production-grade "meet-in-the-middle" architecture bridging asymptotic mathematical optimization with modern CPU cache constraints.

WHAT WAS ADDED

- **Crossover Threshold (Cache Blocking):** Introduced a hardware-optimized base case limit (threshold = 64). Once the matrix quadrants shrink to this size, Strassen recursion completely halts and delegates the remaining work to cache-friendly normal multiplication.
- **Smart Micro-Padding:** Instead of padding up to massive powers of 2, added a calculated padding wrapper that only expands the matrix enough to be a clean multiple of the crossover threshold.
- **Top-Level Parallel Branching:** Dispatched the 7 primary Strassen branches to execute simultaneously across 7 independent CPU cores.

WHAT WAS REMOVED

- **Addition Bloat:** Entirely eliminated the millions of deep-level matrix additions and subtractions that choke the pure Strassen method.
- **Memory Waste:** Removed the excessive memory allocation caused by blind power-of-2 padding.
- **Fringe Peeling:** Deliberately discarded complex "matrix peeling" (edge-slicing) logic to ensure the CPU always receives perfectly square, predictable memory blocks for uninterrupted L1/L2 cache execution.

Results

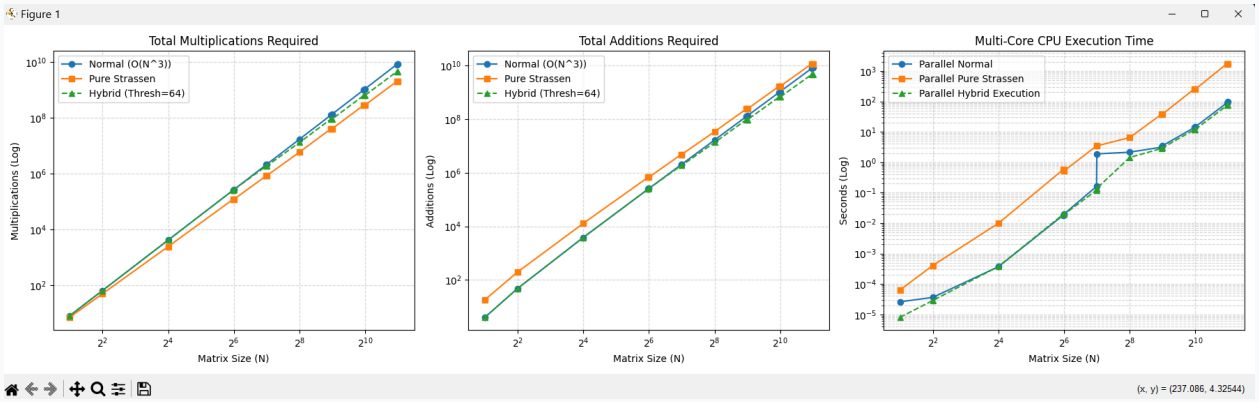


Figure 1: Benchmark performance scaling of matrix multiplication implementations.